Introduktion till C++

Uppgift 2 – Refaktorera kasinot

Uppgiftens målsättning

Att introducera **referenser** och tankar om struktur i C++-projekt. Vidare är målsättningen att den studerande introduceras till en iterativ utvecklingsprocess och hur refaktorering är en del av kodens kvalitetssäkring. Den studerande får öva sig i att se nya möjligheter öppnas upp av förändringar i förutsättningarna.

Uppgift

Det huvudsakliga målet med uppgiften är att applicera refaktorering (*refactoring*) för att förbättra strukturen från tidigare uppgifter. Fokus för uppgiften är att, med *uppgift 1* som utgångspunkt, applicera ny kunskap för att främja vidare arbete i projektet. Det uppnås genom att förenkla, strukturera och förbättra koden med:

- **struct** för att **gruppera data**
- **enum** för att **öka läsbarheten**
- Bli av med hårdkodade värden som bör vara konstanter
- samt **uppdelning** i **filer** och **namespaces** för att förbättra strukturen och läsbarheten för projektet

Det innebär **inte** att originalkoden var felaktig eller dålig utan för det mesta är det ett resultat av att förutsättningarna har förändrats sedan koden ursprungligen skrevs och för att främja möjligheten att underhålla projektet.

I uppgiften, utöver refaktoreringen, skall även ett nytt spelbord läggas till i spelet.

Refaktorering

Vårt kasino är i behov av refaktorering då vi har lärt oss nya saker som möjliggör enklare och elegantare kod. Men även på grund av att förutsättningarna har förändrats. När vi började skriva koden var problemet mycket mindre. Vi skulle bara göra ett litet projekt med ett spel. Isolerat krävdes inte att vi tog hänsyn till möjligheten **återanvända**, **underhålla** eller **utöka** projektet. Sedan dess har troligtvis lagt till:

- Återanvändbarhet där vi anropar samma funktion i olika situationer
- Underhåll där vi förändrat befintlig funktionalitet
- Utökat med menyer, pengar osv

Projekt växer, ny funktionalitet tillkommer, befintlig kod och förutsättningar förändras. Därför behöver vi ibland gå tillbaka och se om vi kan strukturera projektet på ett bättre sätt, ett sätt som gör det enklast möjligt att underhålla och vidareutveckla det.

Viktiga saker vi har lärt oss är **funktionsargument**, **referenser** och **structar**. Det gör att vi kan hantera state för data mer dynamiskt jämfört med att endast ha tillgång till lokala och globala variabler. Globala variabler kan göra det svårt att se och förstå beroenden. Skickas data istället som referenser kan det vara lättare att förstå livslängd och flödet i ett program. Vi ska också använda namespaces och flera filer för att strukturera programmet.

I denna uppgift görs förändringen i fyra steg:

Steg 1
Utnyttja structar, referenser och funktioner för att ta bort **alla** globala och statiska variabler.

Steg 2
Använd enums där det är möjligt, till exempel för:

- Menyval
- Där integer används för att representera något (till exempel 1=Gul, 2=Röd, 3=Blå)

Steg 3
Ta bort alla hårdkodade värden (magic numbers) och ersätt dem med konstanter.

Steg 4
Gruppera ihop kod som hör ihop och separera den från övrig kod för att lättare hitta i projektet och få en bättre översikt. Detta gör även att det är lättare att uppdatera och felsöka programmet.

För att uppnå detta skall namespaces användas. Det tillåter oss att binda flera funktioner och structar under ett specifikt namn (vårt namespace). Till exempel kan vi göra **ett namespace per spelbord** och kanske ytterligare ett för **gemensamma funktioner**. Menyerna kan också ligga i ett eget namespace – var och en får känna efter vad som fungerar bäst. **Varje namespace ska ligga i en egen fil.**

Funktionsdeklarationerna ska ligga i en h-fil och deras definitioner i en cpp-fil med samma namn som headern de deklarerats i. Använd `#include` för att inkludera headerfilerna där de behövs.

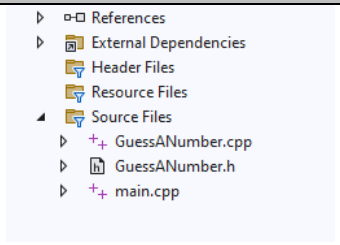
Exempel:

main.cpp

```cpp
#include <iostream>
#include "GuessANumber.h"

int main()
{
    int money = 1000;
    std::cout << "Casino Game\n";
    Guess_A_Number::PlayGame(money);

}
```



GuessANumber.h

```cpp
namespace Guess_A_Number
{
    void PlayGame(int& someMoney);
}
```

GuessANumber.cpp

```cpp
#include <iostream>

namespace Guess_A_Number
{
    void PlayGame(int& someMoney)
    {
        std::cout << "Welcome to Guess A Number" << std::endl;
        std::cout << "Hello, you have $" << someMoney << " with you." << std::endl;

        return;
    }
}
```

Fler spel

Nu då systemet är modulärt och väl uppdelat med interface emellan (headerfilerna) borde det vara en enkel sak att lägga till ytterligare spel. Det ska vi testa genom att lägga till ett nytt spel (det totala antalet spel är därefter fyra spel). Vilket spel ni väljer är valfritt utöver att det **inte** ska vara något av de spel som redan finns eller roulett.

Inlämning

Gör mapp med uppgiftens namn. I denna mapp skapar ni ytterligare tvåmappar. En som heter **Exe** och en som heter **Source**.

I **Source**-mappen innehåller all källkod samt projekt filer.
Exe-mappen ska endast **filer** som behövs för att köra programmet ligga. Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

För att lösa uppgiften är följande verktyg, eller koncept tillåtna. En del av er kanske redan har kunskaper i C++ och vet andra sätt att lösa uppgiften med hjälp av andra koncept. Det är inte tillåtet, uppgiften kan och skall lösas endast med angivna verktyg!

```
#include, #pragma once
assert
funktioner
namespace
char, short, int, long, float, double, bool, void, unsigned
const, constexpr static
struct
enum class
std::array, referenser, pekare
typedef
if, else, else if, switch, while, do, for, return, break
random_device, mt19937, uniform_int_distribution
cout, cin
static_cast<>, const_cast<>
system(“cls”), system(“pause”)
```